Nexus interview Based Q&A

1. What is Nexus Repository?
Definition:
Nexus Repository is an artifact repository manager used to store, manage, and distribute software artifacts.
It can store artifacts such as Maven packages, npm packages, Python packages, NuGet packages, and Docker images.
It provides a central location from where CI/CD pipelines and developers can upload and download artifacts.
It also supports access control, repository management, and caching of artifacts from external repositories.
Simple meaning:
Nexus ni manam application build artifacts ni **central place lo store and manage** cheyadaniki use chestham.
For example, Jenkins application ni build chesina tarvata generated artifact or Docker image ni Nexus lo store cheyyachu.
Later deployment time lo same artifact ni Nexus nunchi download chesi DEV, QA, and PROD environments lo deploy cheyyachu.
Simple flow:
Developer
↓
Jenkins
↓
Build Artifact
↓
Nexus Repository
↓
Download
↓
Deploy
Main point:
Nexus Repository = **Central place to store, manage, and distribute application artifacts.**

---

2. What is the difference between Hosted, Proxy, and Group repositories?
Definition:
A Hosted repository is a repository where our organization stores and manages its own artifacts.
A Proxy repository connects to an external repository and caches artifacts locally when they are requested.
A Group repository provides a single endpoint that combines multiple repositories, usually hosted and proxy repositories.
These repository types help organizations manage internal artifacts and external dependencies in a controlled way.
Simple meaning:
Hosted → Our own artifacts
Proxy → External artifacts cached locally
Group → Single endpoint for multiple repositories
Example:
Suppose our Java application needs:
Company Artifact → Hosted Repository
Maven Central Dependency → Proxy Repository
Application access → Group Repository
A developer can configure Maven to use the Group repository instead of configuring multiple repository URLs separately.
Main point:
Hosted = **Store our artifacts**
Proxy = **Cache external artifacts**
Group = **Combine repositories behind one endpoint**

---

3. How do you upload an artifact to Nexus?
Definition:
Uploading an artifact to Nexus means publishing a generated application package into a Nexus repository.
The artifact is normally created during the application build process using tools such as Maven, Gradle, npm, or other package managers.
The build tool is configured with the Nexus repository URL and the required authentication details.
After successful authentication and validation, the artifact is uploaded to the appropriate hosted repository.
Simple meaning:
First application ni build chestham.
Build ayina artifact ni Nexus ki publish chestham.
Simple flow:
Source Code
↓
Build
↓
Artifact Created
↓
Authenticate with Nexus
↓
Upload Artifact
↓
Nexus Hosted Repository
Example:

For a Maven application:
mvn clean package
mvn deploy
Here, mvn deploy configured repository ki artifact ni publish chestundi.
Main point:
Build → **Authenticate** → Upload → **Nexus**

---

4. Docker image push to Nexus is failing. What will you check?
Definition:
When a Docker image push to Nexus fails, I first verify whether the Nexus Docker registry endpoint is reachable.
Then I check Docker authentication and whether the user has permission to push images into that repository.
I also verify the image name, registry address, repository configuration, and image tag.
If these are correct, I check network connectivity and Nexus logs to identify the exact failure.
Simple meaning:
Docker image push fail ayithe direct ga retry cheyyakunda step-by-step ga check chestha.
Troubleshooting flow:
Nexus Availability
↓
Docker Registry URL
↓
Authentication
↓
Repository Permission
↓
Image Name / Tag
↓
Network
↓
Nexus Logs
Example:
docker login nexus.example.com
docker tag myapp:v1 nexus.example.com/myapp:v1
docker push nexus.example.com/myapp:v1
Main point:
First **Nexus → Login → Permission → Image/Tag → Network → Logs** check chestha.

---

5. Nexus is running but artifacts cannot be downloaded. What will you check?
Definition:
If Nexus is running but artifacts cannot be downloaded, I first verify that the required repository is available and healthy.
Then I check whether the requested artifact and version actually exist in the repository.
I verify the user's read permissions and authentication configuration.
Finally, I check network connectivity, repository configuration, and Nexus logs for the exact error.
Simple meaning:
Nexus service running undadam alone sufficient kadu. Repository, artifact, permissions, and connectivity kuda correct undali.
Troubleshooting flow:
Nexus Service
↓
Repository
↓
Artifact / Version
↓
Authentication
↓
Read Permission
↓
Network
↓
Logs
Example:
For a Docker image:
docker pull nexus.example.com/myapp:v1
If it fails, I check whether myapp:v1 actually exists and whether the user has permission to pull it.
Main point:
Service running ≠ Artifact access guaranteed.

---

6. Nexus disk usage is very high. What will you check?
Definition:
High disk usage in Nexus can occur because repositories contain many old artifacts, Docker images, or cached dependencies.
I first identify which filesystem and repository are consuming the most storage.
Then I review old versions, unused artifacts, Docker image layers, and repository retention requirements.
I use approved cleanup policies and retention rules instead of manually deleting important production artifacts.

Simple meaning:
Disk full avuthunte random ga files delete cheyyakudadhu.
First:
Disk Usage
↓
Which Repository?
↓
What is consuming storage?
↓
Old / Unused Artifacts
↓
Cleanup Policy
↓
Controlled Cleanup
Useful commands:
df -h
du -sh <directory>
Important point:
Nexus repository data ni manually delete cheyyadam avoid chestha. Nexus-supported cleanup and retention mechanisms use chestha.
Main point:
Check → Identify → Review → Cleanup safely

---

7. How do you manage Nexus artifacts across DEV, QA, and PROD?
Definition:
I manage artifacts across environments by using versioned and traceable artifacts instead of rebuilding the application separately for each environment.
The CI pipeline creates the artifact once and publishes it to Nexus with a unique version.
The same tested artifact is then promoted through DEV, QA, and PROD environments.
This ensures that the artifact tested in QA is the same artifact deployed to production.
Simple meaning:
Application ni every environment kosam separate ga build cheyyakunda **Build Once** approach follow chestha.
Simple flow:
Source Code
↓
Jenkins Build
↓
Versioned Artifact
↓
Nexus
↓
DEV
↓
QA
↓
PROD
Suppose:
myapp:1.5.0
QA lo test chesina 1.5.0 artifact ne PROD lo deploy chestham.
Main point:
Build Once → Test → Promote Same Artifact

---

8. How do you integrate Nexus with Jenkins?
Definition:
Jenkins can integrate with Nexus to publish build artifacts and retrieve artifacts required during deployment.
During the build stage, Jenkins compiles and packages the application and publishes the generated artifact to Nexus.
Jenkins uses securely managed credentials to authenticate with Nexus.
During deployment, Jenkins can retrieve the required versioned artifact from Nexus and deploy it to the target environment.
Simple meaning:
Jenkins
↓
Build
↓
Artifact
↓
Nexus
↓
Download Required Version
↓
Deploy

For example, Jenkins Maven application ni build chesi generated JAR ni Nexus lo upload chestundi.
Deployment stage lo same version JAR ni Nexus nunchi retrieve chesi application server/container ki deploy cheyyachu.
Main point:
Jenkins = **Build & Deploy**
Nexus = **Store & Distribute Artifacts**

---

9. Nexus authentication is failing. What will you check?
Definition:
Nexus authentication failure means the client cannot successfully authenticate with the Nexus server.
I first verify whether the configured username and password, token, or other authentication method is correct.
Then I check whether the user account is active and has the required repository permissions.
I also verify that the client is connecting to the correct Nexus endpoint and that no proxy or network issue is interfering with authentication.
Simple meaning:
Login fail ayithe credentials matrame check cheyyakunda account and permissions kuda check chestha.
Troubleshooting flow:
Nexus URL
↓
Username / Token
↓
Account Status
↓
Repository Permission
↓
Network / Proxy
For Docker:
docker login nexus.example.com
Main point:
Credentials → Account → Permission → Endpoint → Network

---

10. Maven build is failing because it cannot download dependencies from Nexus. What will you check?
Definition:
When Maven cannot download dependencies from Nexus, I first verify the repository configuration used by Maven.
Then I check whether Maven is pointing to the correct Nexus Group or Proxy repository.
I verify Nexus availability, authentication, repository permissions, and whether the required dependency is available.
If the repository is acting as a proxy, I also check its connection to the upstream external repository.
Simple meaning:
Maven dependency download fail ayithe Maven configuration nunchi Nexus varaku complete path check chestha.
Troubleshooting flow:
Maven Configuration
↓
Nexus URL
↓
Group / Proxy Repository
↓
Authentication
↓
Dependency
↓
Upstream Repository
Useful commands:
cat ~/.m2/settings.xml
mvn dependency:tree
mvn clean install
Main point:
Maven → Nexus → Repository → Dependency → Upstream

---

11. Nexus Proxy repository is not downloading artifacts from the internet. What will you check?
Definition:
A Nexus Proxy repository retrieves artifacts from an external upstream repository and can cache them locally.
If it cannot download artifacts, I first verify the configured upstream repository URL.
Then I check whether the Nexus server has outbound network access, DNS resolution, and any required proxy configuration.
I also check the Nexus proxy repository status and logs to identify authentication, connectivity, or upstream errors.
Simple meaning:
Nexus Proxy repository ki external internet repository tho connection undali.
Troubleshooting flow:
Nexus
↓
Proxy Repository
↓
DNS Resolution

↓
Network / Proxy
↓
External Repository
For example, Maven Central reachability ni test cheyyadaniki:
curl -I https://repo1.maven.org
nslookup repo1.maven.org
Main point:
Upstream URL → DNS → Network → Proxy → Nexus Logs

---

12. Same artifact version is being overwritten. How will you handle it?
Definition:
Artifact overwriting means the same version identifier is being used for multiple different builds.
For production releases, I prefer immutable versioning so that a released version cannot be replaced by another build.
Each release should have a unique version that allows us to identify exactly which artifact was tested and deployed.
This improves traceability, rollback capability, and release reliability.
Simple meaning:
Oka version ki oka exact artifact undali.
Bad approach:
myapp:1.0
↓
Build 1

myapp:1.0
↓
Build 2
I prefer:
myapp:1.0.1
myapp:1.0.2
myapp:1.0.3
Every version represents a specific build.
Main point:
One Release Version → One Immutable Artifact

---

13. How do you secure Nexus?
Definition:
Nexus security means protecting stored artifacts, repository access, credentials, and administrative operations from unauthorized users.
I use role-based access control so users receive only the permissions required for their responsibilities.
I use HTTPS for secure communication and securely manage authentication credentials and tokens.
I also restrict administrative access, review permissions regularly, and follow the principle of least privilege.
Simple meaning:
Andariki full access ivvakunda, required access matrame provide chestha.
Security approach:
HTTPS
↓
Authentication
↓
RBAC
↓
Least Privilege
↓
Repository Permissions
↓
Admin Restriction
↓
Regular Access Review
For example:
Developer → Read / Publish as required
CI User → Required repository write access
Admin → Administrative access
Main point:
Secure Communication + Authentication + RBAC + Least Privilege

---

14. Nexus is down. What will you check?
Definition:
If Nexus is down, I first confirm the impact and verify whether the Nexus service or container is actually unavailable.
Then I check server resources such as disk space, memory, CPU, and filesystem availability.
I review Nexus application logs and storage/database-related health information to identify the root cause.
After resolving the issue, I verify that users and CI/CD pipelines can successfully upload and download artifacts.

Simple meaning:
Production lo Nexus down ayithe first service status, resources, logs, storage, and connectivity check chestha.
Troubleshooting flow:
Impact
↓
Nexus Service
↓
CPU / Memory / Disk
↓
Logs
↓
Storage / Database Health
↓
Network
↓
Artifact Upload / Download Test
Depending on how Nexus is deployed, useful checks may include:
systemctl status nexus
df -h
free -m
Main point:
Service → Resources → Logs → Storage → Network → Verify

---

15. How do you handle Nexus backup and recovery?
Definition:
Nexus backup and recovery is the process of protecting repository data and configuration so that the repository can be restored after a failure.
The backup strategy should cover the data required to recover Nexus according to the organization's architecture and backup policy.
Backups should be stored safely and protected from accidental deletion or corruption.
Most importantly, restoration should be tested periodically because having a backup does not guarantee that recovery will work.
Simple meaning:
Backup teesukovadam alone enough kadu. **Restore test** kuda important.
Simple flow:
Nexus Data
↓
Backup
↓
Secure Storage
↓
Recovery Test
↓
Restore
↓
Verify Artifacts
Recovery test lo:
Nexus Restore
↓
Repository Available
↓
Artifact Download
↓
Artifact Upload
↓
Application Verification
Main point:
Backup → Secure Storage → Restore Test → Recovery Verification

---

16. CI pipeline successfully built the artifact but Nexus upload failed. What will you check?
Definition:
If the CI pipeline builds the artifact successfully but cannot upload it to Nexus, I first separate the build problem from the repository publishing problem.
Since the build succeeded, I focus on Nexus availability, repository URL, authentication, and write permissions.
Then I verify the artifact version and repository configuration used by the pipeline.
I also check network connectivity and Nexus logs before retrying the upload.
Simple meaning:
Build → SUCCESS
Nexus Upload → FAILED
So build ni again troubleshoot cheyyakunda upload path ni focus chestha.
Troubleshooting flow:
Nexus Availability
↓

Repository URL
↓
Credentials
↓
Write Permission
↓
Artifact Version
↓
Network
↓
Nexus Logs
Main point:
Build OK → Focus on Nexus Publishing

---

17. Docker image exists in Nexus but Kubernetes cannot pull it. What will you check?
Definition:
If Kubernetes cannot pull a Docker image from Nexus, I first verify that the image repository name and tag in the Pod specification are correct.
Then I verify that the Kubernetes nodes can reach the Nexus Docker registry endpoint.
If the registry requires authentication, I check whether the appropriate image pull secret is configured and referenced correctly.
I also inspect Pod events and container runtime errors to determine whether the problem is authentication, networking, DNS, TLS, or the image itself.
Simple meaning:
Nexus lo image undadam alone sufficient kadu. Kubernetes node ki Nexus registry ni access cheyyagalige connectivity and authentication undali.
Troubleshooting flow:
Image Name / Tag
↓
Nexus Registry
↓
Network / DNS
↓
TLS / Certificate
↓
imagePullSecret
↓
Pod Events
Useful command:
kubectl describe pod <pod_name> -n prod
If image pull authentication is required, I also verify the relevant secret configuration.
Main point:
Image → Registry → Network → Authentication → Pod Events

---

18. How do you troubleshoot Nexus in a Production environment?
Definition:
In a production Nexus issue, I first understand the impact and identify whether artifact upload, download, or both are affected.
Then I check Nexus availability, system resources, repository health, authentication, network connectivity, and application logs.
I avoid making uncontrolled changes because Nexus may be a critical dependency for multiple CI/CD pipelines.
After identifying the root cause, I apply the safest fix, verify artifact access, and document the issue and resolution.
Simple meaning:
Production issue lo random changes cheyyakunda **impact → investigation → fix → verification** approach follow chestha.
Troubleshooting flow:
Production Impact
↓
Nexus Availability
↓
Resources
↓
Repository Health
↓
Authentication / Network
↓
Logs
↓
Root Cause
↓
Fix
↓
Verification
Main point:
Investigate carefully → Fix safely → Verify completely

19. How do you promote artifacts from DEV to PROD?
Definition:
Artifact promotion means moving the same tested artifact through different environments without rebuilding it for every environment.
The CI pipeline creates a versioned artifact and stores it in Nexus.
That artifact is deployed to DEV, tested, and then promoted to QA and PROD after the required approvals and validations.
Using the same artifact across environments ensures that the artifact tested earlier is the one eventually deployed to production.
Simple meaning:
DEV lo test chesina artifact ni rebuild cheyyakunda same artifact ni PROD ki promote chestham.
Simple flow:
Build Once
↓
Nexus
↓
DEV
↓
Testing
↓
QA
↓
Approval
↓
PROD
For example:
myapp:2.4.0
DEV lo test chesina 2.4.0 artifact ne QA and PROD lo use chestham.
Main point:
Build Once → Test → Promote Same Artifact → PROD

---

20. Explain a real-time CI/CD scenario using Jenkins, Nexus, and Kubernetes.
Definition:
In a typical CI/CD setup, Jenkins automates the process of building, testing, packaging, and deploying an application.
Nexus acts as the central repository for storing the versioned application artifact or container image produced by the pipeline.
Kubernetes runs the application using the exact version of the image that was built and stored in Nexus.
This approach provides traceability because we can identify which version was built, tested, and deployed.
Simple meaning:
Developer code push chesina daggara nunchi Kubernetes deployment varaku complete flow ila untundi.
Simple flow:
Developer
↓
Git
↓
Jenkins
↓
Build & Test
↓
Docker Image
↓
Nexus
↓
Kubernetes
↓
Deployment
↓
Verification
Suppose application version:
myapp:v1.2.0
Jenkins:
Build → Test → Docker Image
Then image Nexus lo store chestundi:
nexus.example.com/myapp:v1.2.0
Kubernetes deployment aa exact image version ni use chestundi.
Deployment verify cheyyadaniki:
kubectl rollout status deployment/myapp
If the new version has a problem, Kubernetes deployment strategy and the organization's rollback process can be used to return to the previous known-good version.
Main point:
Git → Jenkins → Build/Test → Nexus → Kubernetes → Verify → Rollback if required